Report Deep Learning Assignment 2

Rahul Chander | Eduard Iorga

Introduction:

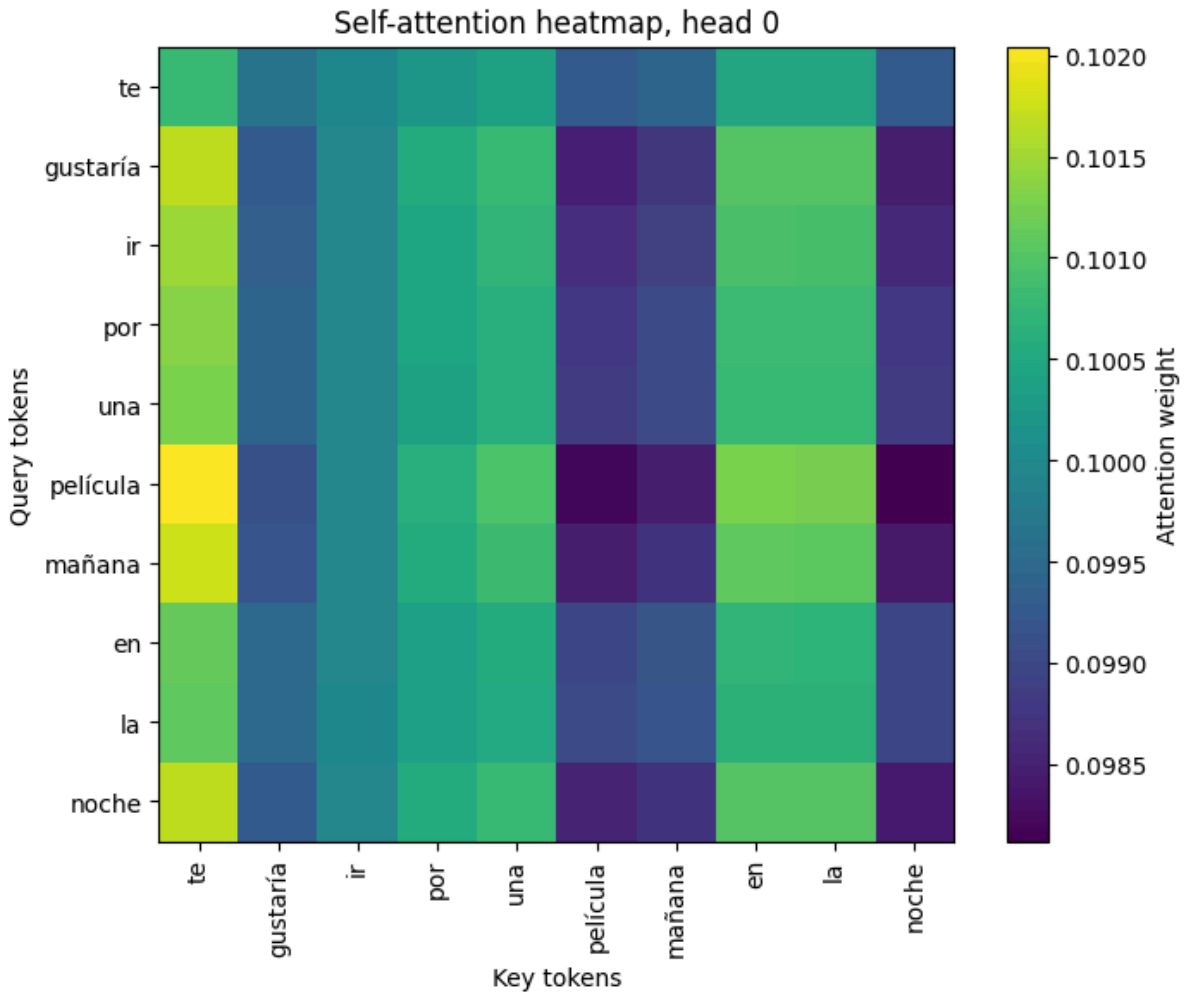
During this assignment we had to do 3 experiments:

- Experiment 0: We were provided with the base code and had to modify it in order to make it work. Furthermore, the base code serves as the baseline to which we will later compare our custom made MultiHeadAttention layer
- Experiment 1: Implement our own MultiHeadAttention layer and read the paper “Attention is all you need” in order to understand how to implement such a layer.
- Experiment 2: Implement visualization for our own MultiHeadAttention layer as well as plotting an attention matrix for one sentence.

During this work, we used Keras to implement the architecture and do the training.

Experiment 0:

In the experiment 0, as we said we were provided a code and we had to run it to obtain some results.
After 30 Epochs, we had a training accuracy of 30.91%, a validation accuracy of 25.87%, a training loss of 0.9815 and a validation loss of 2.2715.
As we can see the model is overfitting, this can be explained by the fact that we did not use regularization techniques, as we trained this model as our baseline for the next experiments.



This plot represent the self-attention with the sentence “¿Te gustaría ir por una película mañana en la noche?”. We can see that in this head, a lot of attention is focused a lot on the first token “te” but also a little bit on the tokens “en”, “la”. An interpretation is that the model is paying attention to the determinants of the sentence.

Experiment 1:

After training the base model we received, we started implementing our own MultiHeadAttention layer. Through implementing it, we put a lot of emphasis on understanding how the queries, keys and values interact, furthermore paying attention to dimensionality, since a good part of the debugging process was due to the dimensionality.

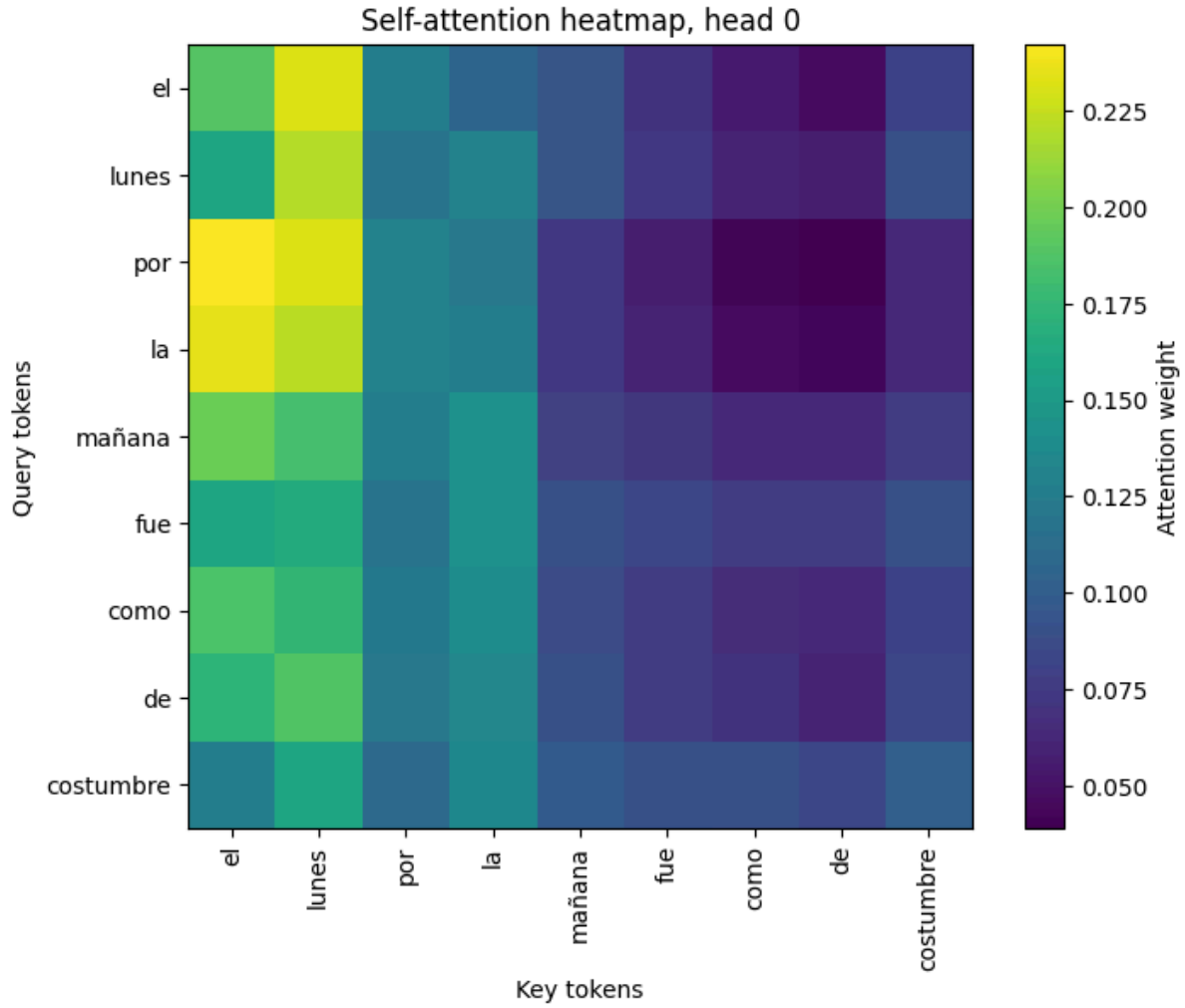
Instead of performing a single attention function over the full embedding dimension, in our case 256, we split the queries, keys, and values into different heads. The reason for this is to allow the model to simultaneously attend to information from different representation subspaces. For example, one head might focus on grammatical structure while another focuses on semantic meaning.

After calculating attention for all heads, the results must be merged back into a single vector that matches the original input shape for the subsequent residual connection. Thus, we transpose and reshape the heads back into the shape (batch, seq_len, embed_dim), resulting in a final dense layer that is applied to the concatenated output.

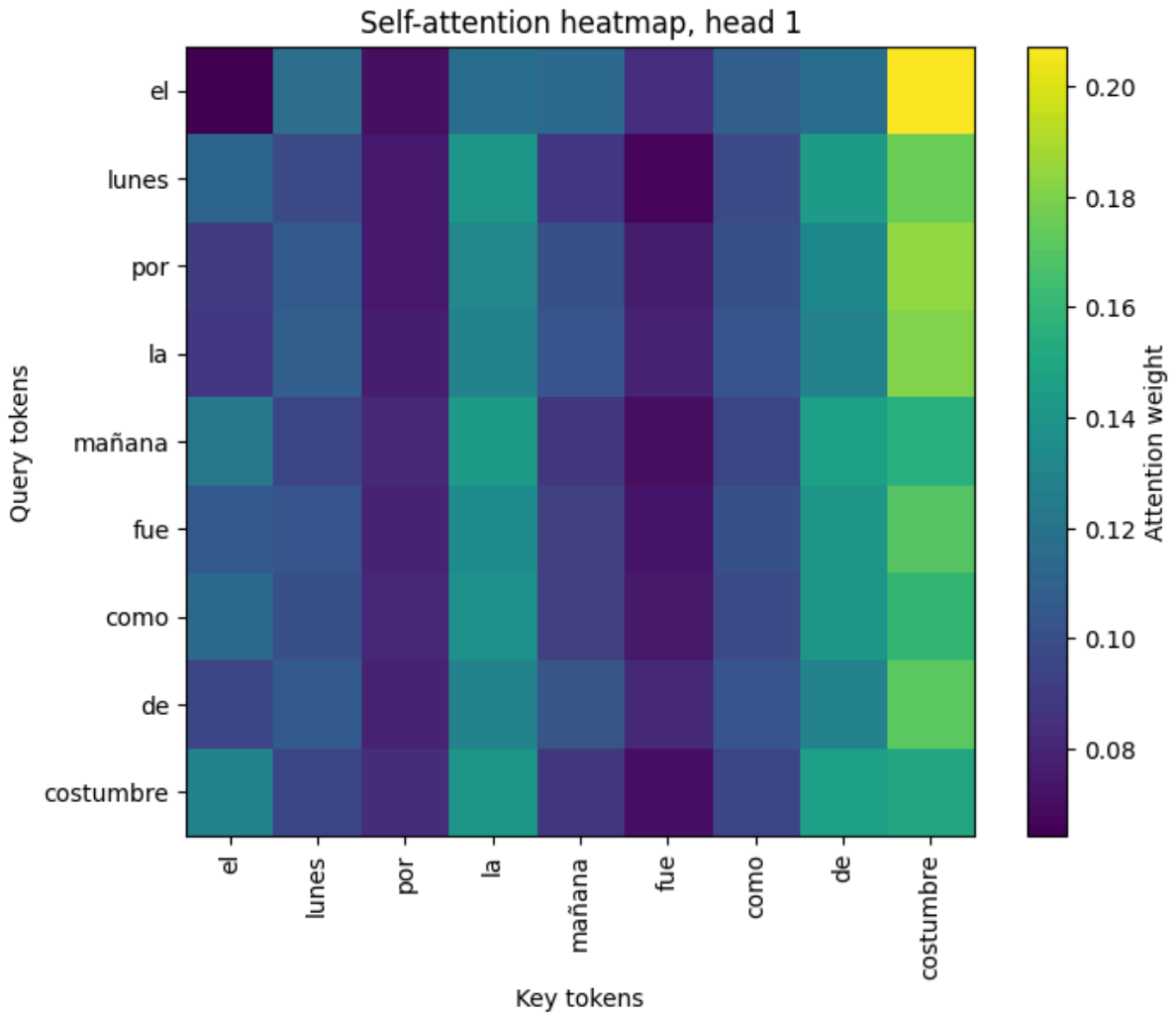
The obtained results are accuracy: 0.3105 - loss: 0.9782 - val_accuracy: 0.2608 - val_loss: 2.2326

Experiment 2:

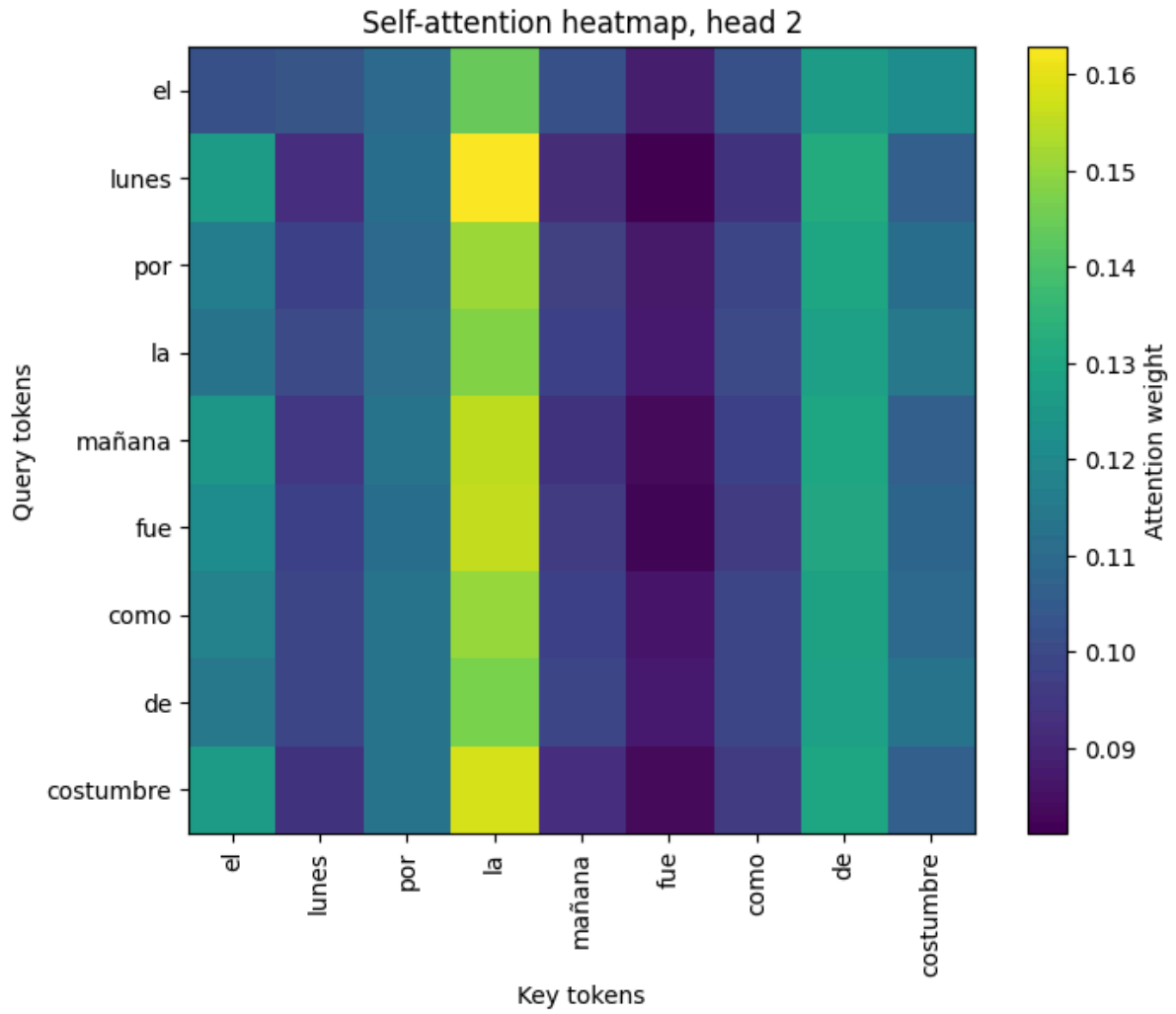
After training the model with the custom self-attention layer, we did again an analysis on the self attention weights of the custom multi head attention layer.
One big thing we could see is that when plotting a few (here 3), we could clearly see what exactly the model is focused on.



In the head 0 of the first custom multi attention layer, we can clearly see that the attention is focused on the 2 first tokens: “el lunes”, which helps the model to understand which token is mostly related with those.



At the opposite, this head is clearly focused on the last token: “costumbre”. Again it helps the model to understand the meaning of specific words in their respective place in the sentence.



In this last head we chose to show, we clearly see that the attention is clearly focused on the token: “la”.

As we see, each attention head can have a different focus, which highlights its capacity to make the model understand the meaning of a specific token in the context of a sentence.

Conclusion

The self-attention mechanism enables the model to focus on different parts of the input sequence, significantly enhancing its contextual comprehension.
We applied this mechanism to a translation task, which is relevant as translation requires to capture both the semantic meaning of a word and its context-dependent nuances.
Our results, visualized through heatmaps plots of the attention heads, demonstrate that the model learned to pay attention to distinct and specific parts of the input, with each head capturing a different relationship.